"Sapienza" Università di Roma
Ingegneria dell'Informazione, Informatica e Statistica
Dipartimento di Informatica

Linguaggi e Compilatori

Autore
Vincenzo Bova

A.A. 2025/2026

Indice

1 Introduzione **2**
1.1 Linguaggi Compilati vs Linguaggi Interpretati 2
1.2 Struttura di un compilatore 3
1.3 Scoping 3
1.4 Passaggio dei parametri 3

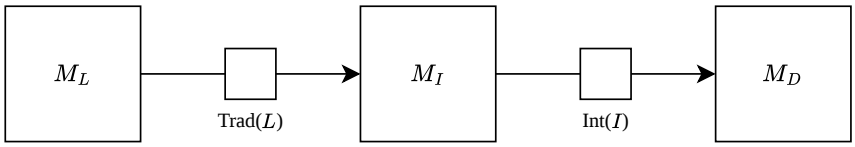
2 Costruzione di un compilatore **4**
2.1 Sintassi 4

1
Introduzione

1.1 Linguaggi Compilati vs Linguaggi Interpretati

In informatica moderna, quasi nessun linguaggio è puramente compilato oppure puramente interpretato. Il codice sorgente non viene mai passato direttamente all'hardware, ma esiste una gerarchia di componenti che collaborano tra loro:

- **Macchina del linguaggio (M_L):** è l'entità in grado di "capire" direttamente programmi in linguaggio L;
- **Trad(L):** prende il codice analizzato da M_L e lo traduce nel linguaggio I;
- **Macchina intermedia (M_I):** è l'entità in grado di "capire" il linguaggio intermedio I;
- **Int(I):** prende il codice analizzato da M_I e lo traduce nel linguaggio D;
- **Macchina di destinazione (M_D):** un tempo era la macchina fisica M_{HW} e D era il set di istruzioni della sua CPU. Con l'avvento dei Sistemi Operativi è diventata M_{SO} e D è composto da linguaggio macchina e chiamate di sistema.

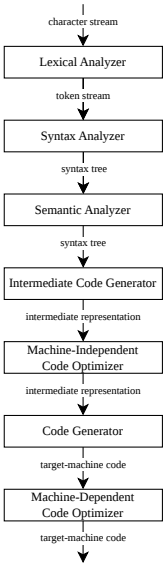


Nella realtà il 99% dei linguaggi ad alto livello hanno M_I significative e i linguaggi puramente compilati o puramente interpretati sono dei casi limite.

1.2 Struttura di un compilatore

Un compilatore può essere suddiviso in due grandi blocchi:

- **Frontend:** si occupa di analizzare il codice sorgente e assicurarsi che sia corretto ed è composto da: *Lexical Analyzer*, *Syntax Analyzer* e *Semantic Analyzer*;
- **Backend:** si occupa di tradurre il programma analizzato in codice eseguibile ed è composto da: *Intermediate Code Generator*, *Machine-Independent Code Optimizer*, *Code Generator* e *Machine-Dependent Code Optimizer*.



In particolare, *Lexical* e *Syntax Analyzers* non lavorano quasi mai in sequenza lineare ma collaborano continuamente, mentre *Semantic Analyzer* e *Intermediate Code Generator* sono spesso fusi assieme. Ogni stadio della compilazione riceve e produce una rappresentazione intermedia, modificando la **Symbol Table**.

1.3 Scoping

Un compito fondamentale del compilatore è gestire la visibilità degli identificatori (variabili, funzioni, etc.) tramite le regole di **scoping**:

- **Scoping statico:** la visibilità di un nome è determinata al tempo di compilazione in base alla struttura del testo;
- **Scoping dinamico:** la visibilità dipende dall'ordine di chiamata delle funzioni a tempo di esecuzione.

1.4 Passaggio dei parametri

Determina come le informazioni vengono trasmesse tra il chiamante e la funzione:

- **Call by value:** viene passata una copia del valore. È sicuro ma può essere inefficiente per strutture grandi;
- **Call by reference:** viene passata la locazione di memoria. Permette effetti collaterali e risparmio di memoria;
- **Call by name:** l'espressione viene valutata solo quando effettivamente usata nella funzione (tipico dei linguaggi funzionali).

2
Costruzione di un compilatore

2.1 Sintassi

Il primo passo per costruire un compilatore prevede di definire formalmente la grammatica del linguaggio.

2.1.1 Grammatiche Context-Free (CFG)

Definizione 2.1: Grammatica Context-Free

Una grammatica context-free è una quadrupla $G = (T, N, P, S)$, dove:

- **Insieme di simboli terminali (T):** simboli elementari non sostituibili;
- **Insieme di simboli non terminali (N):** variabili sintattiche sostituibili tramite produzioni;
- **Insieme di produzioni (P):** regole di riscrittura nella forma $A \to \alpha$;
- **Simbolo di partenza (S):** il punto di inizio della derivazione, con $S \in N$.

A questo punto, data una stringa contenente almeno un non terminale, possiamo scegliere un non terminale contenuto al suo interno e sostituirlo mediante una delle produzioni definite nella grammatica.

Definizione 2.2: Derivazione

Data una CFG $G = (T, N, P, S)$, una **derivazione** è una sequenza di stringhe $\alpha_0, \alpha_1, \ldots, \alpha_k$ t.c. $\forall 0 \leq i < k$:

1. $\alpha_i = \gamma_i A_i \delta_i$;
2. $A_i \to \theta_i \in P$;
3. $\alpha_{i+1} = \gamma_i \theta_i \delta_i$.

In tal caso diciamo che α_k è derivabile da α_0, $\alpha_0 \overset{*}{\Rightarrow} \alpha_k$.

Definizione 2.3: Linguaggio di una CFG

Data una CFG $G = (T, N, P, S)$, il suo **linguaggio** $L(G)$ è definito come:

$$L(G) = \{u \in T^* \mid S \overset{*}{\Rightarrow} u\}$$

2.1.2 Parse Tree

Definizione 2.4: Parse Tree

Data una CFG $G = (T, N, P, S)$ un **parse tree** in G è un **albero radicato** tale che:

- la radice è S;
- ogni foglia è un terminale o ϵ;
- Ogni nodo interno è un non-terminale;
- Se A è un nodo interno con figli $X_1, \ldots, X_n$ allora $A \to X_1 \ldots X_n \in P$.

Teorema 2.1: Corrispondenza tra parse tree e $L(G)$

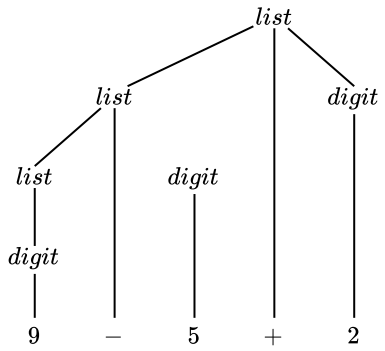
Siano $G = (T, N, P, S)$ una CFG, T un parse tree in G e u una stringa:

$$u \in L(G) \iff \exists \text{ PT per } u$$

In particolare, T è una parse tree per u se le foglie di T concatenate da sinistra a destra formano u.

Esempio:

$$list \to list + digit \mid list - digit \mid digit$$
$$digit \to 0 \mid 1 \mid \ \ldots \ \mid 9$$

Derivazione per $9 - 5 + 2$:

1. $list$
2. $list + digit$
3. $list - digit + digit$
4. $digit - digit + digit$
5. $9 - digit + digit$
6. $9 - 5 + 2$